K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

UNIT 2 ASSIGNMENT

LDST - Linear Data Structures Toolkit

Data Structures (ETCCDS202)

Name of the School:	School of Engineering & Technology
Program:	B.Tech CSE (AI and ML)
Course Title:	Data Structures
Course Code:	ETCCDS202
Unit Title:	Linear Data Structures Toolkit (LDST)
Student Name:	GAURI SHARMA
Roll Number:	2501730108
Section:	A
Semester:	2
Batch:	2025-26
Submitted To:	Mrs. Neetu Chauhan
Submission Date:	

ASSIGNMENT OVERVIEW:

This project, titled the Linear Data Structures Toolkit (LDST), involves implementing fundamental data structures—Dynamic Arrays, Linked Lists, Stacks, and Queues—from scratch using Python. It focuses on manual memory management through resizing logic and pointer manipulation. The toolkit concludes with a practical application: a stack-based algorithm to validate balanced parentheses in strings.

Part A: Dynamic Array

- **What is a Dynamic Array?**

A dynamic array is a data structure that can change its size during runtime. Unlike normal arrays, it does not have a fixed size. When the array becomes full, it automatically increases its capacity.

- **How append() works**

When we insert a new element:

1. If the array has space → element is added directly.
2. If the array is full →

A new array of double size is created

All elements are copied

Then the new element is inserted

- **Amortized Time Complexity**

Most append operations take O(1) time because we are simply adding an element at the end. Sometimes resizing happens, and copying takes O(n) time.

However, resizing does not happen frequently. Therefore, when we average the cost of all operations.

Amortized Time Complexity = O(1)

Part B: Singly Linked List (Operations + Concept)

- **What is a Singly Linked List?**

A singly linked list is a linear data structure where each element (node) contains:

- Data
- Address of the next node

The elements are not stored in continuous memory locations.

Operations Implemented

1. Insert at Beginning
   A new node is created and placed at the start.
   Time Complexity: O(1)

2. Insert at End
   Traverse till the last node and attach the new node.
   Time Complexity: O(n)

3. Delete by Value
   Search for the value and remove the node.
   Time Complexity: O(n)

4. Traversal
   Visit each node one by one.
   Time Complexity: O(n)

Advantages of Linked List

- ✓ No need to shift elements
- ✓ Dynamic size
- ✓ Easy insertion and deletion

Part C: Doubly Linked List

- **What is a Doubly Linked List?**

A doubly linked list is similar to a singly linked list, but each node contains:

- Data
- Pointer to next node
- Pointer to previous node

Operations Implemented

1. Insert at End
   Adds a node at the last position.
   Time Complexity: O(n)

2. Insert After a Node
   Inserts a new node after a given value.
   Time Complexity: O(n)

3. Delete at Position
   Deletes a node at a specific position.
   Time Complexity: O(n)

Advantage over Singly Linked List

• Traversal in both directions • Easier deletion using previous pointer

---

Part D: Stack using Linked List (LIFO Concept)

- **What is a Stack?**

A stack is a linear data structure that follows the Last In First Out (LIFO) principle.

Operations Implemented

1. Push
   Adds an element at the top.
   Time Complexity: O(1)

2. Pop
   Removes the top element.
   Time Complexity: O(1)

3. Peek
   Returns the top element without removing it.
   Time Complexity: O(1)

Why Stack Operations are O(1)?
All operations are performed at the head of the linked list.
No traversal is required, so time remains constant.

Part E: Queue using Linked List (FIFO Concept)

- **What is a Queue?**

A queue follows the First In First Out (FIFO) principle.

Operations Implemented

1. Enqueue
   Adds element at the rear.
   Time Complexity: O(1)

2. Dequeue
   Removes element from the front.
   Time Complexity: O(1)

3. Front
   Returns the front element.
   Time Complexity: O(1)

Why Queue is Efficient?

Using both head and tail pointers ensures that insertion and deletion happen in constant time.

Part F: Balanced Parentheses Checker (Application)

Problem Statement

Check whether a given expression has balanced brackets.

Approach Using Stack

1. Traverse the expression
2. Push opening brackets
3. On closing bracket → pop and match
4. If mismatch occurs → not balanced
5. If stack is empty at end → balanced

Examples

- "([])" → Balanced
- "([)]" → Not Balanced
- "((" → Not Balanced
- "" → Balanced

Time Complexity

- Traversing string → O(n)
- Stack operations → O(1)

Overall Complexity = O(n)

Part G: Conclusion

- In this assignment, I implemented different linear data structures such as dynamic arrays, linked lists, stack, and queue.
- I understood how these data structures work internally and how they are used to solve real-world problems like expression checking.
- This assignment also helped me understand time complexity and efficiency of different operations.